Engineering Portfolio

Evan Chiu

Firmware and Embedded Software

NYCU M.S. student, Mechanical Engineering, Control Division

C, RISC-V RVV, Linux, MATLAB, Python

github.com/huaien418

RISC-V RVV computation strategies

Problem

Reduce the cost of 8×8 DCT and IDCT while tracking differences in numerical results.

My contribution

Implemented and analyzed Parts 1–3: scalar C, RVV reduction and independent outputs. My teammate owned CUDA.

C and RVV assembly
Tools: gem5, Docker, Makefile

01 Scalar baseline

```
sum += a[i] * b[i]
```

Nested loops establish a comparison baseline

02 RVV reduction

```
vle32, vfmul, vfredusum.vs
```

Vector products reduce into one accumulated result

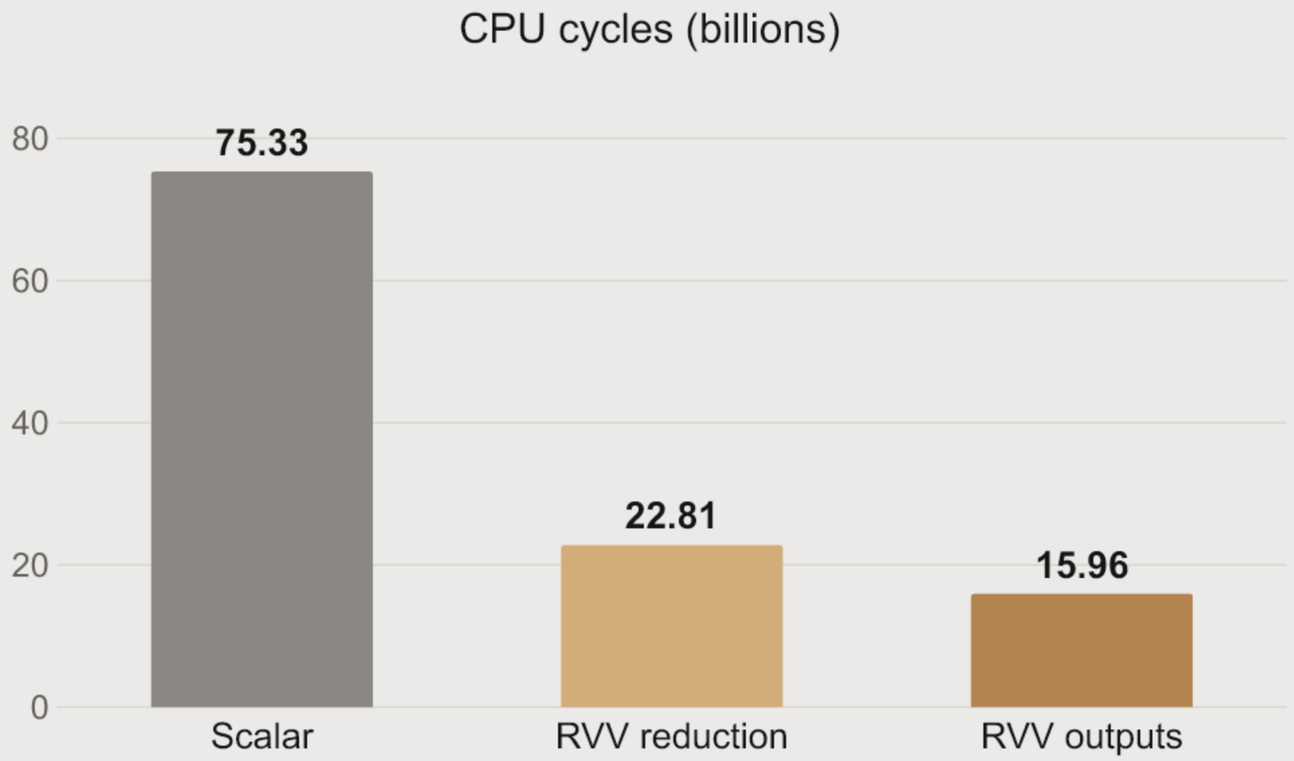
03 Independent RVV outputs

```
vlse32, vfadd, vse32
```

Each lane accumulates a separate output

Scope: DCT, quantization and image reconstruction. Full JPEG bitstream encoding is outside this project.

RVV performance and numerical results



gem5 23.1.0.0, TimingSimpleCPU, VLEN 256
5184×3456 input, 279,936 blocks

4.72×

Scalar cycles ÷ Part 3 cycles

5.84×

Scalar instructions ÷ Part 3 instructions

Archived output comparison

Part 1 and Part 3 PGM files are byte-identical.
Part 2 differs from Part 1 at 5,826 pixels.

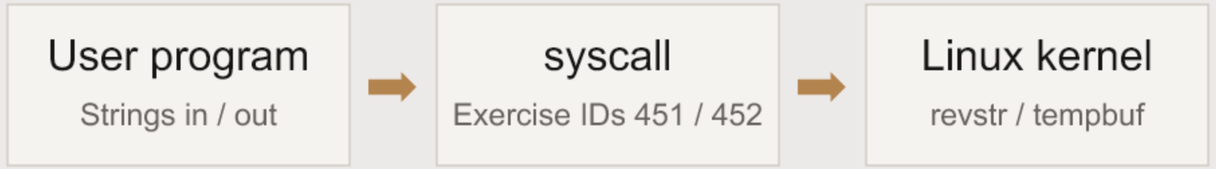
Performance comes from archived gem5 statistics. Image checks cover saved outputs. The simulator was not rerun.

Archived gem5 results and numerical checks

Linux system call implementation

Goal and environment

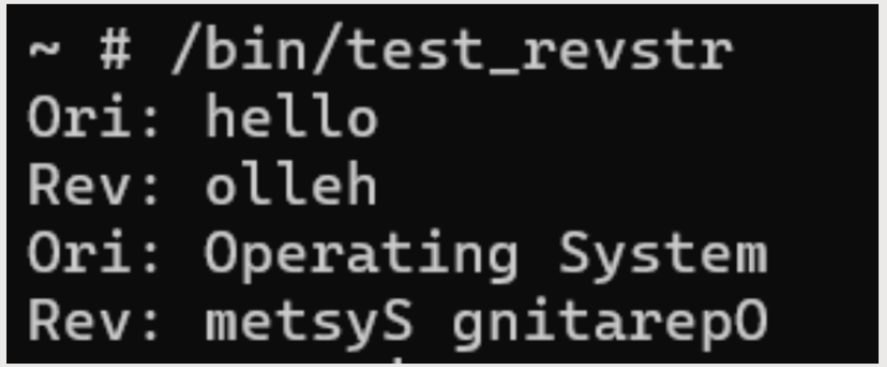
Added string reversal and a kernel string list to a RISC-V Linux 6.1 course kernel. The original environment used QEMU.



Kernel patch includes registration, declarations and Kbuild wiring

Implementation

revstr copies, reverses and returns a string. tempbuf supports ADD, REMOVE and PRINT. Used uaccess helpers, a linked list and mutex-protected list operations.



Historical course output: string reversal

Historical evidence retained. No fresh kernel build or QEMU run. Concurrency and length boundaries remain unverified.

pthread scheduling and synchronization

Experiment

Compared NORMAL and FIFO policies and priorities by observing worker execution order.

CPU affinity

Requested CPU0 affinity for the experiment

Start barrier

Released workers through a shared barrier

CPU-time work

Measured busy work using per-thread CPU time

Order in the historical case

FIFO 30 (Thread 2) → FIFO 10 (Thread 1) → NORMAL (Thread 0)

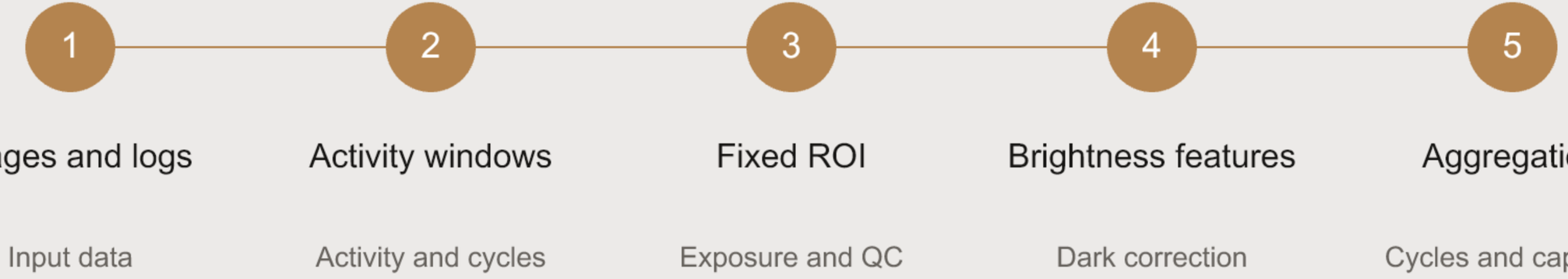
```
/root # ./sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30
Thread 2 is running
Thread 2 is running
Thread 2 is running
Thread 1 is running
Thread 1 is running
Thread 1 is running
Thread 0 is running
Thread 0 is running
Thread 0 is running
```

Original report image. Order depends on policy, privileges and runtime conditions.

Scheduling tests were not rerun. Affinity success and return-value handling remain validation items.

SLM image analysis and validation

Organize scan images into traceable brightness features, quality checks and repeatability results.



Research and software work

Analysis workflow, feature definitions and interpretation. Code development and review used AI assistance, disclosed in the repository.

Published synthetic checks

30	240	5
datasets	feature rows	plots

Synthetic checks use a test double and fixed windows. Thickness detection, synchronization and power compensation remain unverified.

Skills and project index

Technical work

C and RISC-V RVV assembly
gem5 profiling and numerical checks
Linux system calls and pthread scheduling
MATLAB analysis and Python practice

Relevant coursework

Operating Systems, Computer Architecture
Algorithms, Image Analysis
Signals and Systems, Digital Control

Independent study and English

CS50P practice: functions, control flow, exceptions and libraries.
TOEIC 910, TOEFL iBT 93

Computer Architecture Project

Parts 1–3, archived metrics and numerical analysis

Operating Systems Projects

System calls, scheduling and kfetch validation status

SLM CircleScan Analysis

Methods, synthetic demo and validation scope

Contact

Evan Chiu

huaienqiu.en14@nycu.edu.tw

github.com/huaien418